

La solution : les outils

Étendre les capacités agentiques

Référence : [Documentation ADK - Qu'est-ce qu'un outil ?](#)

Extrait de la documentation ADK :

"Dans le contexte d'ADK, un outil représente une fonctionnalité spécifique fournie à un agent IA pour lui permettre d'effectuer des actions et d'interagir avec le monde au-delà de ses capacités de base de génération de texte et de raisonnement."

Les outils offrent de nombreuses possibilités :

-  **Accéder aux informations en temps réel** : recherche sur le Web, API de météo, cours de bourse
-  **Effectuer des calculs** : exécuter du code, lancer des modèles financiers, traiter des données
-  **Interroger des bases de données** : récupérer les commandes de clients, l'inventaire de produits et les profils utilisateur
-  **Interagir avec des systèmes externes** : envoyer des e-mails, prendre des rendez-vous, traiter des paiements
-  **Effectuer des actions dans le monde réel** : actualiser des enregistrements, déclencher des workflows, contrôler des appareils

Exemple simple :

```
def get_weather(city: str) -> dict:
    """Récupère les conditions météorologiques actuelles dans une ville donnée."""
    # Dans une implémentation réelle, une API de météo serait appelée
    return {
        "status": "success",
        "report": f"Dans la ville de {city}, le temps est ensoleillé et la température est de 25 °C."
    }

# Cette fonction Python devient un outil lorsqu'elle est ajoutée à un agent
```

Concepts fondamentaux

1. Qu'est-ce qu'un outil ?

Référence : [Documentation ADK - Qu'est-ce qu'un outil ?](#)

Extrait de la documentation ADK :

"Techniquement parlant, un outil est généralement un composant de code modulaire, comme

une fonction Python, une méthode de classe ou même un autre agent spécialisé, conçu pour exécuter une tâche distincte et prédéfinie."

Principales caractéristiques :

Axé sur l'action : les outils effectuent des actions spécifiques pour un agent, qu'il s'agisse de rechercher des informations, d'appeler une API ou d'exécuter des calculs.

Étend les capacités agentiques : les outils permettent aux agents d'accéder à des informations en temps réel, d'influencer des systèmes externes et de dépasser les limites de connaissances inhérentes à leurs données d'entraînement.

Exécute une logique prédéfinie : point crucial, les outils exécutent une logique spécifique définie par le développeur. Ils ne possèdent pas de capacités de raisonnement qui leur sont propres, contrairement au LLM de base de l'agent. Le LLM raisonne (quel outil utiliser, quand et avec quelles entrées) alors que l'outil se contente d'exécuter sa fonction désignée.

Exemple d'outil simple :

```
def get_capital_city(country: str) -> str:
    """Récupère la capitale d'un pays donné."""
    capitals = {
        "france": "Paris",
        "japan": "Tokyo",
        "canada": "Ottawa"
    }
    return capitals.get(
        country.lower(),
        f"Désolé, je ne connais pas la capitale du pays suivant : {country}."
    )
```

Pourquoi est-ce un outil ?

- Il s'agit d'une **fonction Python** (composant de code modulaire).
- Il effectue une tâche **spécifique** (rechercher les capitales).
- Il possède une **logique prédéfinie** (le dictionnaire des capitales).
- Il **étend les capacités de l'agent** (accès aux données structurées).

2. Comment les agents utilisent les outils

Référence : [Documentation ADK - Comment les agents utilisent les outils](#)

Extrait de la documentation ADK :

"Les agents exploitent les outils de manière dynamique grâce à des mécanismes impliquant souvent des appels de fonction. Le processus suit généralement ces étapes :

1. **Raisonnement** : le LLM de l'agent analyse ses instructions système, l'historique de la conversation et la requête de l'utilisateur.

2. **Sélection** : à partir de cette analyse, le LLM décide quel outil exécuter, le cas échéant, en fonction des outils disponibles pour l'agent et des docstrings qui décrivent chaque outil.
3. **Appel** : le LLM génère les arguments (entrées) requis pour l'outil sélectionné et déclenche son exécution.
4. **Observation** : l'agent reçoit la sortie (résultat) renvoyée par l'outil.
5. **Finalisation** : l'agent intègre la sortie de l'outil dans son processus de raisonnement en cours pour formuler la prochaine réponse, décider de l'étape suivante ou déterminer si l'objectif a été atteint."

Exemple d'enchaînement :

```
L'utilisateur demande : "Quel temps fait-il à Paris ?"
↓
Étape 1 - Raisonnement :
  L'agent analyse : "L'utilisateur a besoin de données météorologiques actuelles
pour Paris."
↓
Étape 2 - Sélection :
  L'agent décide : "L'outil get_weather répond à ce besoin."
↓
Étape 3 - Appel :
  L'agent appelle : get_weather(city="Paris")
↓
Étape 4 - Observation :
  L'outil renvoie : {"status": "success", "report": "Soleil à Paris, 20 °C"}
↓
Étape 5 - Finalisation :
  L'agent répond : "Le temps est actuellement ensoleillé à Paris, et la température
est de 20 °C."
```

Insight clé : Le LLM gère le raisonnement et la prise de décision, tandis que l'outil gère l'action et la récupération des données. C'est cette séparation qui donne du pouvoir aux agents.

Représentation visuelle :

```
graph TD
  A[Requête utilisateur] --> B[1. Raisonnement :<br/>L'agent analyse la requête.]
  B --> C{2. Sélection :<br/>Quel outil ?}
  C -->|Outil nécessaire| D[3. Appel :<br/>Appeler l'outil avec des arguments]
  C -->|Aucun outil nécessaire| E[Générer une réponse directe]
  D --> F[4. Observation :<br/>Recevoir le résultat de l'outil]
  F --> G[5. Finalisation :<br/>Formuler la réponse]
  G --> H[Réponse de l'agent]
  E --> H

  style B fill:#e1f5ff
  style D fill:#ffeb99
  style F fill:#e1f5ff
  style G fill:#eeffee
```

Processus en cinq étapes utilisé par les agents pour exploiter les outils

3. Types d'outils dans ADK

Référence : [Documentation ADK - Types d'outils dans ADK](#)

Extrait de la documentation ADK :

"ADK offre une grande flexibilité, car il prend en charge plusieurs types d'outils :

1. **Outils de fonction** : outils créés par vous, adaptés aux besoins spécifiques de votre application.
2. **Outils intégrés** : outils prêts à l'emploi fournis par le framework pour les tâches courantes. Exemples : recherche Google, exécution de code, génération augmentée par récupération (RAG).
3. **Outils tiers** : intégrez facilement des outils provenant de bibliothèques externes populaires."

Cette leçon d'introduction est consacrée à plusieurs sujets :

1. Outils de fonction personnalisés

- Fonctions Python que vous écrivez
- Adaptés à votre logique métier spécifique
- Exemple : `calculate_shipping_cost`, `lookup_order_status`
- **Traités dans** : partie 3

2. Outils intégrés

- Fournis par le framework ADK
- Prêts pour la production, gérés par Google
- Exemples : recherche Google, exécution de code
- **Traités dans** : partie 2

3. Agent-as-tool (agent comme outil)

- Utiliser un autre agent spécialisé comme outil
- Permet la délégation et le raisonnement spécialisé
- Exemple : L'agent principal appelle un agent spécialisé dans l'assistance technique.
- **Présenté dans** : partie 4 ; **couverture complète** : leçons à venir

Tableau comparatif :

Type	Complexité	Configuration	Cas d'utilisation
Intégré	Faible	Importer et utiliser	Besoin de capacités courantes (recherche, exécution de code)

Type	Complexité	Configuration	Cas d'utilisation
Outils de fonction	Moyenne	Écrire une fonction Python	Besoin d'une logique métier personnalisée
Agent-as-tool	Modérée/Élevée	Créer un agent spécialisé	Besoin d'un raisonnement spécialisé pour les sous-tâches

4. Paramètre **tools**

Référence : [Documentation ADK - Équiper l'agent : outils](#)

Extrait de la documentation ADK :

"**tools** (facultatif) : fournissez une liste d'outils que l'agent peut utiliser. Chaque élément de la liste peut être :

- Une fonction ou méthode native (encapsulée en tant que **FunctionTool**)
- Une instance d'une classe héritant de **BaseTool**
- Une instance d'un autre agent (**AgentTool**)"

Comment le LLM appréhende les outils :

"Le LLM utilise les noms des fonctions/outils, les descriptions (issues des docstrings ou du champ de description) et les schémas de paramètres pour déterminer quel outil appeler en fonction de la conversation et de ses instructions."

Exemple de code :

```
from google.adk.agents import LlmAgent

# Étape 1 : Définir une fonction d'outil
def get_capital_city(country: str) -> str:
    """Récupère la capitale d'un pays donné."""
    capitals = {"france": "Paris", "japan": "Tokyo", "canada": "Ottawa"}
    return capitals.get(country.lower(), f"Désolé, je ne connais pas la capitale du pays suivant : {country}.")

# Étape 2 : Ajouter l'outil à l'agent
capital_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="capital_agent",
    description="Répond aux questions sur les capitales.",
    instruction="Tu es un agent qui donne la capitale d'un pays.",
    tools=[get_capital_city] # ✨ NOUVEAU : paramètre "tools"
)
```

Points à noter :

1. **"Tools" est une liste** : `tools=[...]` - vous pouvez fournir plusieurs outils.
2. **Fonction passée directement** : ADK encapsule automatiquement les fonctions Python en tant que `FunctionTool`.
3. Le **LLM utilise des métadonnées** : le nom de la fonction, la docstring et les paramètres guident la décision du LLM.
4. **Pas d'encapsulation manuel nécessaire** : Python ADK gère automatiquement la création de `FunctionTool`.

Comment le LLM voit cet outil ?

```
Nom de l'outil : get_capital_city
Description : "Récupère la capitale d'un pays donné."
Paramètres :
  - country (str) : nom d'un pays
Renvoie : str
```

Le LLM utilise ce schéma pour décider quand et comment appeler l'outil.

5. Outils et état de la session (aperçu)

Lien avec la leçon 4 : Gestion de l'état et de la mémoire

Les outils peuvent fonctionner de concert avec l'état de la session pour fournir un comportement personnalisé et contextuel. Les mécanismes d'accès à l'état dans les outils (`tool_context`) seront abordés par la suite, dans les leçons portant sur les rappels et le contrôle, mais voici un aperçu de ce qu'il est possible de faire :

Exemple : Outil qui enregistre les préférences utilisateur dans l'état

```
def save_user_preference(preference_key: str, preference_value: str, ctx) -> dict:
    """Enregistre une préférence utilisateur dans l'état de la session.

    Args:
        preference_key : le nom de la préférence (par exemple, "langue", "thème")
        preference_value : la valeur de préférence (par exemple, "espagnol", "sombre")
        ctx : contexte de l'outil (donne accès à l'état de la session)

    Renvoie :
        dict : message d'état

    Remarque : Le paramètre ctx (tool_context) est traité dans la leçon 6.
    """
    # Enregistrer dans l'espace de noms user: pour assurer la persistance d'une
    session à l'autre
    ctx.session.state[f"user:{preference_key}"] = preference_value

    return {
```

```
"status": "success",  
"message": f"Saved {preference_key} = {preference_value}"  
}
```

Fonctionnalités proposées :

-  **Personnalisation** : les outils peuvent consulter les préférences utilisateur à partir de l'état.
-  **Contexte** : les outils peuvent vérifier l'état de la conversation précédente.
-  **Workflows multi-étapes** : les outils peuvent enregistrer les résultats intermédiaires dans l'état.
-  **Mémoire d'une session à l'autre** : les outils peuvent utiliser l'espace de noms `user` : pour assurer la persistance.

Exemple de cas d'utilisation :

```
# Tour 1 : l'utilisateur définit sa préférence  
Utilisateur : "Enregistre l'espagnol dans mes préférences de langue."  
L'agent appelle : save_user_preference("language", "Spanish", ctx)  
# Maintenant state["user:language"] = "Spanish"  
  
# Tour 2 : l'agent utilise la préférence (grâce au modèle d'état de la leçon 4)  
Instruction de l'agent : "Réponds en {user:language?English}"  
# Résolution : "Réponds en espagnol"
```

Aperçu : L'intégration complète des outils avec la gestion d'état est abordée dans la leçon 6, où vous découvrirez `tool_context`, le paramètre `ctx` et les modèles de coordination d'état avancés.

Exemple pratique

Créer votre premier agent utilisant des outils

Objectif : Créer un assistant de géographie qui aide les utilisateurs à découvrir les capitales des différents pays à l'aide d'un outil personnalisé.

Étape 1 : Créer le projet

```
adk create geography_assistant  
cd geography_assistant
```

Fonction :

- Crée un répertoire de projet ADK
- Établit la structure de base des agents
- Crée `agent.py` avec le code d'agent par défaut

Étape 2 : Écrire l'agent

Remplacez le contenu de `agent.py` par :

```
"""
Agent assistant de géographie
Montre le fonctionnement du paramètre "tools" d'ADK à l'aide d'un simple outil de
fonction personnalisé.

Référence : https://google.github.io/adk-docs/agents/llm-agents#tools
"""

from google.adk.agents import LlmAgent

# Étape 1 : Définir une fonction d'outil
def get_capital_city(country: str) -> str:
    """Récupère la capitale d'un pays spécifié.

    Args:
        country (str) : le nom du pays.

    Renvoie :
        str : le nom de la capitale ou le message d'erreur.
    """
    # Base de données simulée de capitales
    capitals = {
        "france": "Paris",
        "japan": "Tokyo",
        "canada": "Ottawa",
        "germany": "Berlin",
        "brazil": "Brasília",
        "australia": "Canberra",
        "india": "New Delhi",
        "mexico": "Mexico City"
    }

    # Rechercher la capitale
    return capitals.get(
        country.lower(),
        f"Désolé, je n'ai pas d'informations sur la capitale du pays suivant : {country}."
    )

# Étape 2 : Créer un agent avec l'outil
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='geography_assistant',
    description='Aide les utilisateurs à en apprendre davantage sur la géographie mondiale.',
    instruction="""
```

Tu es un assistant de géographie qui aide les utilisateurs à découvrir les capitales du monde.

Lorsqu'un utilisateur pose une question sur une capitale :

1. Utilise l'outil `get_capital_city` pour trouver la réponse.
2. Fournis les informations de manière conviviale et pédagogique.
3. Tu peux ajouter des faits intéressants si tu en connais.

Si l'outil renvoie un message d'erreur, indique poliment à l'utilisateur que tu ne disposes pas de cette information.

```
"""  
tools=[get_capital_city] # Fournir la fonction en tant qu'outil  
)
```

Tutoriel du code :

Lignes 10 à 28 : définir la fonction d'outil

- Le nom de la fonction est descriptif : `get_capital_city`
- Contient des indications de type : `country: str -> str`
- Inclut une docstring expliquant l'objectif et les paramètres
- Renvoie une chaîne simple (nom de la capitale ou message d'erreur)

Lignes 30 à 44 : créer l'agent

- Utilise `LlmAgent` (de la leçon 2–3)
- Ajoute un nouveau paramètre `tools` avec notre fonction
- Les instructions indiquent à l'agent quand et comment utiliser l'outil
- Explique comment gérer les cas de réussite et d'erreur

Étape 3 : Exécuter et tester

adk web

Accédez à <http://localhost:8000> dans votre navigateur.

Étape 4 : Scénarios de test

Test 1 : Demande de capitale de base

Vous : `Quelle est la capitale du Japon ?`

Comportement attendu :

- L'agent appelle `get_capital_city(country="Japan")`.
- L'outil renvoie `"Tokyo"`.

- L'agent répond : "La capitale du Japon est Tokyo."

Points à noter :

- L'agent a automatiquement sélectionné le bon outil.
 - L'agent a correctement extrait "Japon" comme paramètre de pays.
 - L'agent a mis en forme le résultat de l'outil en langage naturel.
-

Test 2 : Plusieurs capitales

Vous : *Dis-moi quelles sont les capitales de la France, de l'Allemagne et du Brésil.*

Comportement attendu :

- L'agent appelle `get_capital_city` trois fois (une fois pour chaque pays).
- L'outil renvoie : "Paris", "Berlin", "Brasilia"
- L'agent fournit une liste mise en forme.

Points à noter :

- L'agent orchestre plusieurs appels d'outils.
 - L'agent comprend qu'il doit utiliser l'outil pour chaque pays.
 - L'agent synthétise les résultats en une réponse cohérente.
-

Test 3 : Capitale inconnue

Vous : *Quelle est la capitale de l'Islande ?*

Comportement attendu :

- L'agent appelle `get_capital_city(country="Iceland")`.
- L'outil répond : "Désolé, je ne dispose d'aucune information concernant la capitale de l'Islande."
- L'agent donne poliment cette réponse à l'utilisateur.

Points à noter :

- L'agent gère les messages d'erreur de façon optimale.
 - Le message d'erreur de l'outil est facile à comprendre.
 - L'agent n'hallucine pas une réponse lorsque les données sont indisponibles.
-

Test 4 : Aucun outil nécessaire

Vous : **Parle-moi de géographie.**

Comportement attendu :

- L'agent répond sans utiliser d'outil.
- Il utilise ses connaissances générales en géographie.
- Il n'appelle aucun outil.

Points à noter :

- Les outils ne sont appelés qu'en cas de besoin.
 - L'agent utilise son raisonnement pour déterminer si un outil est nécessaire.
 - Toutes les requêtes ne nécessitent pas d'outil.
-

Points à retenir

Concepts fondamentaux :

- Les **outils étendent les capacités agentiques** au-delà des connaissances du LLM, ce qui leur permet d'effectuer des actions concrètes et d'accéder à des données réelles.
- **Les agents passent par cinq étapes pour utiliser les outils** : raisonnement → sélection → appel → observation → finalisation.
- **Trois principaux types d'outils** : outils intégrés (prêts à l'emploi), outils de fonction (personnalisés), agent-as-tool (délégation)

Comment fonctionnent les outils ?

- Le **LLM sélectionne les outils à partir** du nom de la fonction, de la docstring et du schéma des paramètres.
- Les **outils ne sont que des fonctions Python** que l'ADK encapsule automatiquement.
- Les **outils renvoient des résultats** que les agents intègrent dans leurs réponses.
- L'**orchestration se fait automatiquement** par le biais de la boucle d'agent d'ADK.

Le paramètre **tools** :

- **Ajouté à LlmAgent** - `tools=[function1, function2, ...]`
- **Les fonctions deviennent automatiquement des outils**. Aucun encapsulage manuel n'est nécessaire en Python.
- Le **LLM décide quand utiliser chaque outil** en fonction du contexte de la conversation et des instructions.

Bonnes pratiques :

- **Noms des fonctions descriptifs** : utilisez `get_capital_city` et non `lookup` ou `get_data`.
- **Docstrings claires** : expliquez ce que fait l'outil et quand il doit être utilisé.
- **Indications de type** : aidez ADK à générer un schéma correct pour le LLM.
- **Valeurs de retour simples** : commencez par les types de base avant les structures

complexes.

Comportement des agents :

- **Les outils sont facultatifs** : les agents n'appellent des outils que lorsque leur raisonnement détermine qu'ils sont nécessaires.
- **Plusieurs appels possibles** : les agents peuvent appeler le même outil plusieurs fois ou des outils différents.
- **Intégration naturelle** : les résultats des outils sont intégrés de manière fluide aux réponses.
- **Gestion des exceptions** : les outils doivent renvoyer des messages d'erreur clairs en cas d'échec.